



课程项目开发规范补充说明（AI时代工程实践要求）

Created 2026年5月13日 09:12

《Linux系统管理与Shell编程》

课程项目开发规范补充说明（AI时代工程实践要求）

本补充说明用于进一步明确：

- AI工具参与开发后的课程要求
- 项目需求分析要求
- 开发过程与版本迭代要求
- 测试与验证要求
- 工程实践能力要求

本说明是对《课程项目开发规范》的补充，不替代原有规范。

一、课程项目评价重点的变化

随着 AI 编程工具的发展：

- 代码生成
- 文档生成
- 架构生成
- 测试模板生成

已经可以在较短时间内完成。

因此：

课程项目的重点不再只是：

“代码是否能够快速生成”

而更加重视：

- 是否发现真实问题
- 是否理解实际需求
- 是否真正经历开发过程
- 是否完成测试与调试
- 是否理解 Linux 系统行为
- 是否能够利用 Linux 解决实际问题

本课程更加关注：

“工程实践能力”

而不仅仅是：

“最终代码结果”。

二、项目必须来源于真实问题

课程项目应从：

“实际问题”

出发，而不是：

“技术关键词堆叠”

例如：

以下描述通常过于空泛：

- 智能运维平台
- 全自动 Linux Agent
- AI 运维系统
- 综合管理平台

这类项目容易出现：

- 功能堆叠

- 目标模糊
- 使用场景不明确
- AI 自动拼装模块

因此：

项目必须能够明确说明：

- 谁在使用系统
 - 遇到了什么问题
 - 为什么现有方法不好用
 - 项目具体解决什么问题
-

三、项目需求分析要求（重要）

项目规划书中必须明确回答以下问题。

1. 项目服务对象是谁？

必须具体。

例如：

- 树莓派开发者
- 实验室管理员
- 小型网站维护人员
- AI训练服务器使用者

不建议使用：

- Linux用户
- 运维人员

等过于宽泛的描述。

2. 实际问题是什么？

问题必须：

- 真实存在

- 可以观察
- 可以验证

例如：

- 日志持续增长导致磁盘占满
- SSH服务异常退出无法及时发现
- 多台设备需要重复执行相同命令

不建议：

- “提高效率”
- “实现智能化”

等抽象目标。

3. 现有方法为什么不好用？

必须说明：

为什么需要开发该项目。

例如：

- Linux原生命令信息不直观
 - 多命令需要重复执行
 - 手工操作容易遗漏
 - 无法自动化运行
-

4. 项目真正解决什么问题？

项目目标必须：

- 明确
- 可测试
- 可验证

例如：

错误描述：

❌ “提升运维效率”

建议描述：

“自动检测磁盘空间异常并生成告警日志”

5. 项目最核心的功能是什么？

项目应重点突出：

1~2个核心功能。

避免：

- 功能过多
- 模块堆叠
- AI自动生成大量无关功能

课程更加重视：

功能是否真正有效，

而不是：

功能数量是否很多。

四、开发过程与版本迭代要求（重要）

课程强调：

“迭代开发过程”

而不是：

“一次性生成完整项目”。

真实工程开发通常需要经历：

- 编写
- 运行
- 出错
- 调试

- 修改
- 再测试

的循环过程。

因此：

项目必须能够体现：

- 功能演进
 - Bug修复
 - 测试验证
 - 方案调整
-

五、Git开发记录要求

项目必须：

- 使用 Git 管理代码
- 保留完整 commit 记录
- 能体现开发过程

commit 应尽量体现：

- 功能增加
- Bug修复
- 测试修改
- 版本演进

不建议：

- 最后一次性提交全部代码
-

六、版本迭代说明（新增重点）

项目需要简要说明：

丨 系统如何从初始版本逐步演进。

建议使用如下形式：

版本	完成内容	发现的问题	修改与优化
v0.1	完成日志统计	grep误报较多	增加关键词过滤
v0.2	增加自动告警	权限不足无法写日志	修改目录权限
v0.3	增加定时任务	crontab无法执行	改用绝对路径

重点在于体现：

- 问题发现
- 调试过程
- 迭代优化
- Linux环境适配

而不仅是：

“最终成功结果”。

七、测试与验证要求（非常重要）

本课程特别强调：

测试与验证能力。

因为：

Linux系统开发中：

真正困难的部分通常不是：

“写代码”，

而是：

- 权限问题
- 环境配置
- Shell调试
- 日志分析
- 服务状态
- 自动化稳定性

因此：

项目必须体现：

- 测试过程
 - 错误分析
 - 修改过程
 - 验证结果
-

八、测试内容建议

项目建议至少包含：

1. 功能测试

验证：

功能是否真正有效。

例如：

- 告警是否触发
 - 日志是否正确生成
 - 自动清理是否成功
-

2. 异常测试

例如：

- 权限不足
- 文件不存在
- 网络断开
- 配置错误

验证：

系统是否能够正常处理异常情况。

3. 边界测试

例如：

- CPU使用率接近100%

- 磁盘空间不足
 - 超大日志文件
-

4. 自动化运行验证

验证：

系统是否能够：

- 自动执行
 - 长时间稳定运行
 - 无人工干预
-

九、课程重点考察的能力

本课程重点考察：

Linux系统实践能力

包括：

- Shell脚本
 - Linux命令使用
 - 系统管理
 - 日志分析
 - 自动化处理
-

工程实践能力

包括：

- 需求分析
 - 问题定义
 - 调试能力
 - 测试能力
 - 版本迭代能力
-

AI时代的软件开发能力

包括：

- 利用AI提高开发效率
 - 判断AI结果是否合理
 - 修改AI生成代码
 - 将AI工具与真实Linux问题结合
-

十、关于AI工具的说明

课程允许使用：

- ChatGPT
- Copilot
- Claude
- Gemini
- Agent工具
- 开源代码

但：

AI工具应作为：

| “开发辅助工具”

而不是：

| “替代整个开发过程”。

课程更关注：

- 是否真正理解系统
 - 是否能够调试与修改
 - 是否能够解决真实问题
 - 是否真正经历开发过程
-

十一、最后说明

本课程项目：

不追求：

- 功能数量最多
- AI术语最复杂
- 架构描述最宏大

更重视：

是否真正解决实际问题

是否真正经历开发与调试过程

是否真正掌握 Linux 系统实践能力。